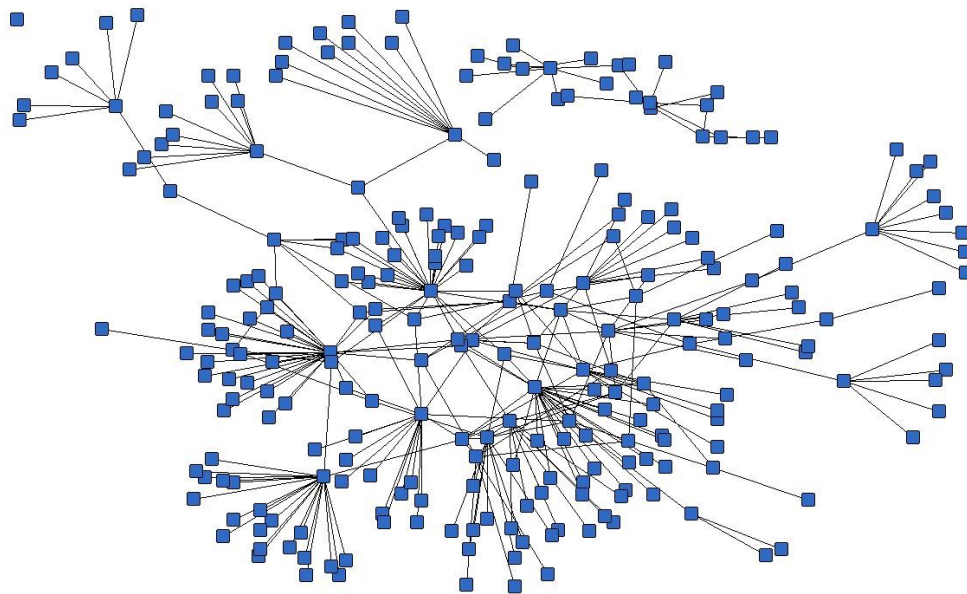


The background of the slide is a solid red color. Overlaid on this is a pattern of small, light red dots. These dots are arranged in a way that they form a large, faint circular shape in the center of the slide, with the density of the dots being higher in the center and fading out towards the edges.

Container networking basics



A dense graph network

Objectives

We will now run network services (accepting requests) in containers.

At the end of this section, you will be able to:

- Run a network service in a container.
- Connect to that network service.
- Find a container's IP address.



Running a very simple service

- We need something small, simple, easy to configure
(or, even better, that doesn't require any configuration at all)
- Let's use the official NGINX image (named `nginx`)
- It runs a static web server listening on port 80
- It serves a default “Welcome to nginx!” page

Running an NGINX server

```
$ docker run -d -P nginx  
66b1ce719198711292c8f34f84a7b68c3876cf9f67015e752b9  
4e189d35a204e
```

- Docker will automatically pull the `nginx` image from the Docker Hub
- `-d / --detach` tells Docker to run it in the background
- `P / --publish-all` tells Docker to publish all ports
(publish = make them reachable from other computers)
- ...OK, how do we connect to our web server now?

Finding our web server port

- First, we need to find the *port number* used by Docker
(the NGINX container listens on port 80, but this port will be *mapped*)

- We can use `docker ps`:

```
$ docker ps
```

CONTAINER ID	IMAGE	...	PORTS	...
e40ffb406c9e	nginx	...	0.0.0.0: `12345`->80/tcp	...

- This means:

port 12345 on the Docker host is mapped to port 80 in the container

- Now we need to connect to the Docker host!



Finding the address of the Docker host

- When running Docker on your Linux workstation:
use `localhost`, or any IP address of your machine
- When running Docker on a remote Linux server:
use any IP address of the remote machine
- When running Docker Desktop on Mac or Windows:
use `localhost`
- In other scenarios (`docker-machine`, local VM...):
use the IP address of the Docker VM



Connecting to our web server (GUI)

Point your browser to the IP address of your Docker host, on the port shown by `docker ps` for container port 80.



Screenshot

You can also use `curl` directly from the Docker host.

Make sure to use the right port number if it is different from the example below:

```
$ curl localhost:12345
<!DOCTYPE html>
<html>
<head>
<title>Welcome to nginx!</title>
...
```

How does Docker know which port to map?

- There is metadata in the image telling “this image has something on port 80”.

- We can see that metadata with `docker inspect`:

```
$ docker inspect --format '{{.Config.ExposedPorts}}' nginx
map[80/tcp:{}]
```

- This metadata was set in the Dockerfile, with the `EXPOSE` keyword.

- We can see that with `docker history`:

```
$ docker history nginx
IMAGE                CREATED              CREATED BY          CMD
7f70b30f2cc6         11 days ago         /bin/sh -c # (nop)
["nginx" "-g" "...
<missing>            11 days ago         /bin/sh -c # (nop)
STOPSIGNAL [SIGTERM]
<missing>            11 days ago         /bin/sh -c # (nop)
EXPOSE 80/tcp
```



Why can't we just connect to port 80?

- Our Docker host has only one port 80
- Therefore, we can only have one container at a time on port 80
- Therefore, if multiple containers want port 80, only one can get it
- By default, containers *do not* get “their” port number, but a random one
(not “random” as “crypto random”, but as “it depends on various factors”)
- We'll see later how to force a port number (including port 80!)



Using multiple IP addresses

Hey, my network-fu is strong, and I have questions...

- Can I publish one container on 127.0.0.2:80, and another on 127.0.0.3:80?
- My machine has multiple (public) IP addresses, let's say A.A.A.A and B.B.B.B. Can I have one container on A.A.A.A:80 and another on B.B.B.B:80?
- I have a whole IPV4 subnet, can I allocate it to my containers?
- What about IPV6?

You can do all these things when running Docker directly on Linux.
(On other platforms, *generally not*, but there are some exceptions.)



Finding the web server port in a script

Parsing the output of `docker ps` would be painful.

There is a command to help us:

```
$ docker port <containerID> 80  
0.0.0.0:12345
```



Manual allocation of port numbers

If you want to set port numbers yourself, no problem:

```
$ docker run -d -p 80:80 nginx
$ docker run -d -p 8000:80 nginx
$ docker run -d -p 8080:80 -p 8888:80 nginx
```

- We are running three NGINX web servers.
- The first one is exposed on port 80.
- The second one is exposed on port 8000.
- The third one is exposed on ports 8080 and 8888.

Note: the convention is `port-on-host:port-on-container`.

Plumbing containers into your infrastructure

There are many ways to integrate containers in your network.

- Start the container, letting Docker allocate a public port for it. Then retrieve that port number and feed it to your configuration.
- Pick a fixed port number in advance, when you generate your configuration. Then start your container by setting the port numbers manually.
- Use an orchestrator like Kubernetes or Swarm. The orchestrator will provide its own networking facilities.

Orchestrators typically provide mechanisms to enable direct container-to-container communication across hosts, and publishing/load balancing for inbound traffic.



Finding the container's IP address

We can use the `docker inspect` command to find the IP address of the container.

```
$ docker inspect --format '{{  
  .NetworkSettings.IPAddress  }}' <yourContainerID>  
172.17.0.3
```

- `docker inspect` is an advanced command, that can retrieve a ton of information about our containers.
- Here, we provide it with a format string to extract exactly the private IP address of the container.

Let's try to ping our container *from another container*.

```
docker run alpine ping `<ipaddress>`  
PING 172.17.0.X (172.17.0.X): 56 data bytes  
64 bytes from 172.17.0.X: seq=0 ttl=64 time=0.106  
ms  
64 bytes from 172.17.0.X: seq=1 ttl=64 time=0.250  
ms  
64 bytes from 172.17.0.X: seq=2 ttl=64 time=0.188  
ms
```

When running on Linux, we can even ping that IP address directly!
(And connect to a container's ports even if they aren't published.)

How often do we use `-p` and `-P` ?

- When running a stack of containers, we will often use Compose
- Compose will take care of exposing containers
(through a `ports :` section in the `docker-compose.yml` file)
- It is, however, fairly common to use `docker run -P` for a quick test
- Or `docker run -p . . .` when an image doesn't EXPOSE a port correctly

We've learned how to:

- Expose a network port.
- Connect to an application running in a container.
- Find a container's IP address.

???

:EN:- Exposing single containers :FR:- Exposer un conteneur isolé



Local development workflow with Docker



Construction site

Objectives

At the end of this section, you will be able to:

- Share code between container and host.
- Use a simple local development workflow.



We want to solve the following issues:

- “Works on my machine”
- “Not the same version”
- “Missing dependency”

By using Docker containers, we will get a consistent development environment.

Working on the “namer” application

- We have to work on some application whose code is at:
<https://github.com/jpetazzo/namer>.

- What is it? We don't know yet!

- Let's download the code.

```
$ git clone https://github.com/jpetazzo/namer
```



Looking at the code

```
$ cd namer  
$ ls -l  
company_name_generator.rb  
config.ru  
docker-compose.yml  
Dockerfile  
Gemfile
```

–

Aha, a `Gemfile`! This is Ruby. Probably. We know this. Maybe?



Looking at the Dockerfile

```
FROM ruby
```

```
COPY . /src
```

```
WORKDIR /src
```

```
RUN bundler install
```

```
CMD ["rackup", "--host", "0.0.0.0"]
```

```
EXPOSE 9292
```

- This application is using a base `ruby` image.
- The code is copied in `/src`.
- Dependencies are installed with `bundler`.
- The application is started with `rackup`.
- It is listening on port `9292`.

Building and running the “namer” application

- Let's build the application with the Dockerfile!

```
$ docker build -t namer .
```

- Then run it. *We need to expose its ports.*

```
$ docker run -dP namer
```

- Check on which port the container is listening.

```
$ docker ps -l
```

Connecting to our application

- Point our browser to our Docker node, on the port allocated to the container.
- Hit “reload” a few times.
- This is an enterprise-class, carrier-grade, ISO-compliant company name generator!

(With 50% more bullshit than the average competition!)

(Wait, was that 50% more, or 50% less? *Anyway!*)

web application 1

web application 1



Making changes to the code

Option 1:

- Edit the code locally
- Rebuild the image
- Re-run the container

Option 2:

- Enter the container (with `docker exec`)
- Install an editor
- Make changes from within the container

Option 3:

- Use a *bind mount* to share local files with the container
- Make changes locally
- Changes are reflected in the container



We will tell Docker to map the current directory to `/src` in the container.

```
$ docker run -d -v $(pwd) :/src -P namer
```

- `-d`: the container should run in detached mode (in the background).
- `-v`: the following host directory should be mounted inside the container.
- `-P`: publish all the ports exposed by this image.
- `namer` is the name of the image we will run.
- We don't specify a command to run because it is already set in the Dockerfile via `CMD`.

Note: on Windows, replace `$(pwd)` with `%cd%` (or `${pwd}` if you use PowerShell).

Mounting volumes inside containers

The `-v` flag mounts a directory from your host into your Docker container.

The flag structure is:

```
[host-path] : [container-path] : [rw|ro]
```

- `[host-path]` and `[container-path]` are created if they don't exist.
- You can control the write status of the volume with the `ro` and `rw` options.
- If you don't specify `rw` or `ro`, it will be `rw` by default.

Hold your horses... and your mounts

- The `-v /path/on/host:/path/in/container` syntax is the “old” syntax
- The modern syntax looks like this:
`--mount
type=bind,source=/path/on/host,target=/path/in/container`
- `--mount` is more explicit, but `-v` is quicker to type
- `--mount` supports all mount types; `-v` doesn't support `tmpfs` mounts
- `--mount` fails if the path on the host doesn't exist; `-v` creates it

With the new syntax, our command becomes:

```
docker run --  
mount=type=bind,source=$(pwd),target=/src -dP namer
```



Testing the development container

- Check the port used by our new container.

```
$ docker ps -l
```

CONTAINER ID	IMAGE	COMMAND	CREATED	STATUS
PORTS		NAMES		
045885b68bc5	namer	rackup	3 seconds ago	Up ...
0.0.0.0:32770->9292/tcp		...		

- Open the application in your web browser.



Making a change to our application

Our customer really doesn't like the color of our text. Let's change it.

```
$ vi company_name_generator.rb
```

And change

```
color: royalblue;
```

To:

```
color: red;
```



Viewing our changes

- Reload the application in our browser.

–

- The color should have changed.

web application 2

web application 2



Understanding volumes

- Volumes are *not* copying or synchronizing files between the host and the container
- Changes made in the host are immediately visible in the container (and vice versa)
- When running on Linux:
 - volumes and bind mounts correspond to directories on the host
 - if Docker runs in a Linux VM, these directories are in the Linux VM
- When running on Docker Desktop:
 - volumes correspond to directories in a small Linux VM running Docker
 - access to bind mounts is translated to host filesystem access (a bit like a network filesystem)



Docker Desktop caveats

- When running Docker natively on Linux, accessing a mount = native I/O
- When running Docker Desktop, accessing a bind mount = file access translation
- That file access translation has relatively good performance *in general* (watch out, however, for that big `npm install` working on a bind mount!)
- There are some corner cases when watching files (with mechanisms like inotify)
- Features like “live reload” or programs like `entr` don’t always behave properly (due to e.g. file attribute caching, and other interesting details!)



Trash your servers and burn your code

(This is the title of a [2013 blog post](#) by Chad Fowler, where he explains the concept of immutable infrastructure.)

- Let's majorly mess up our container.
(Remove files or whatever.)
- Now, how can we fix this?
- Our old container (with the blue version of the code) is still running.
- See on which port it is exposed:
`docker ps`
- Point our browser to it to confirm that it still works fine.



Immutable infrastructure in a nutshell

- Instead of *updating* a server, we deploy a new one.
- This might be challenging with classical servers, but it's trivial with containers.
- In fact, with Docker, the most logical workflow is to build a new image and run it.
- If something goes wrong with the new image, we can always restart the old one.
- We can even keep both versions running side by side.

If this pattern sounds interesting, you might want to read about *blue/green deployment* and *canary deployments*.



Recap of the development workflow

1. Write a Dockerfile to build an image containing our development environment. (Rails, Django, ... and all the dependencies for our app)
2. Start a container from that image. Use the `-v` flag to mount our source code inside the container.
3. Edit the source code outside the container, using familiar tools. (vim, emacs, textmate...)
4. Test the application. (Some frameworks pick up changes automatically. Others require you to Ctrl-C + restart after each modification.)
5. Iterate and repeat steps 3 and 4 until satisfied.
6. When done, commit+push source code changes.



Debugging inside the container

Docker has a command called `docker exec`.

It allows users to run a new process in a container which is already running.

If sometimes you find yourself wishing you could SSH into a container: you can use `docker exec` instead.

You can get a shell prompt inside an existing container this way, or run an arbitrary process for automation.



docker exec example

\$ # You can run ruby commands in the area the app is running and more!

```
$ docker exec -it <yourContainerId> bash
```

```
root@5ca27cf74c2e:/opt/namer# irb
```

```
irb(main):001:0> [0, 1, 2, 3, 4].map {|x| x **  
2}.compact
```

```
=> [0, 1, 4, 9, 16]
```

```
irb(main):002:0> exit
```



Stopping the container

Now that we're done let's stop our container.

```
$ docker stop <yourContainerID>
```

And remove it.

```
$ docker rm <yourContainerID>
```

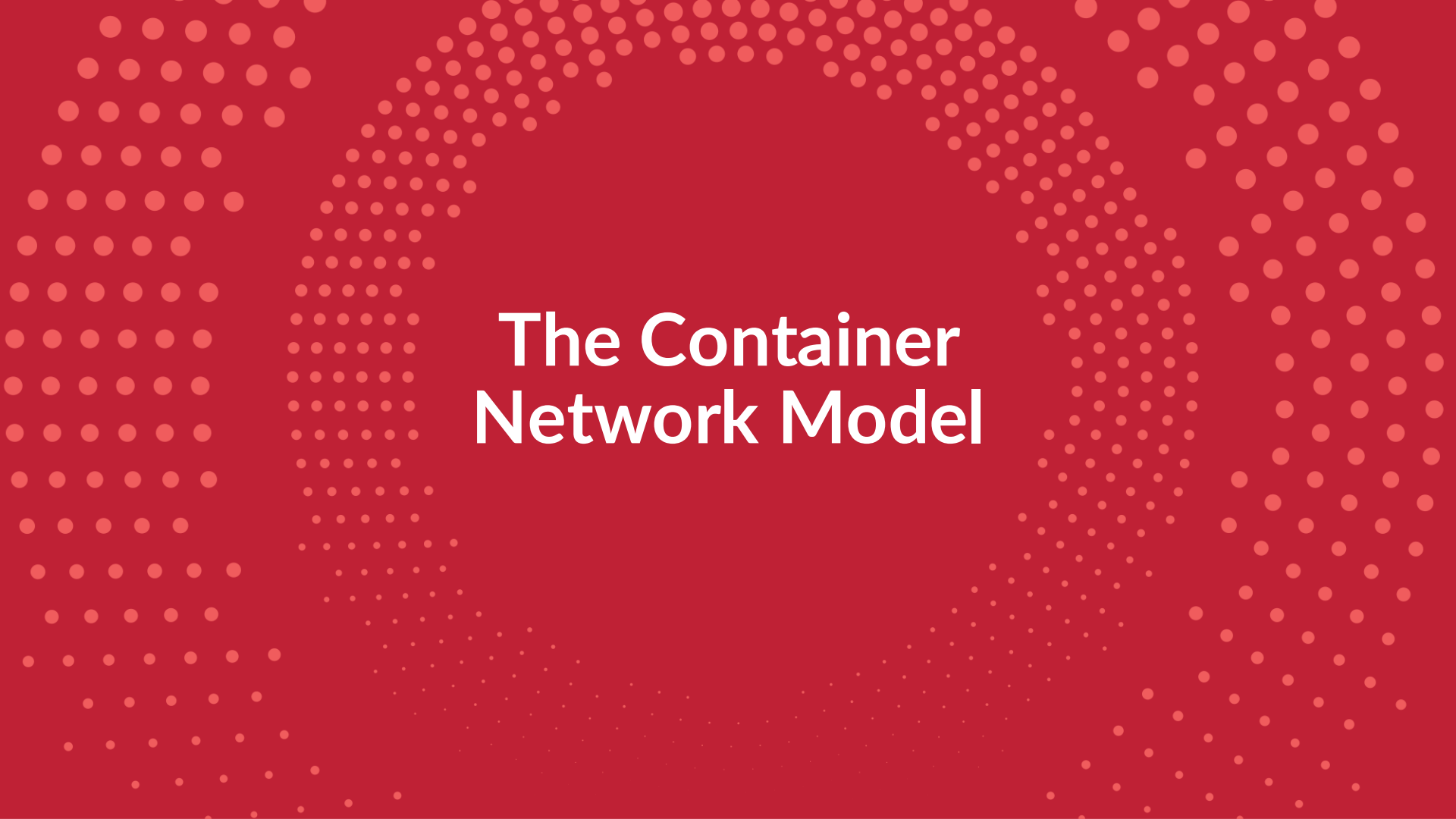


Section summary

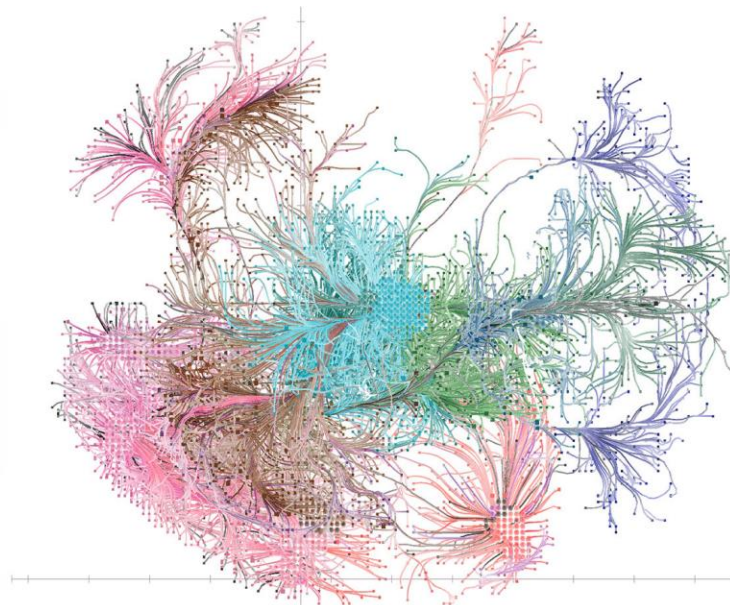
We've learned how to:

- Share code between container and host.
- Set our working directory.
- Use a simple local development workflow.





The Container Network Model



A denser graph network

We will learn about the CNM (Container Network Model).

At the end of this lesson, you will be able to:

- Create a private network for a group of containers.
- Use container naming to connect services together.
- Dynamically connect and disconnect containers to networks.
- Set the IP address of a container.

We will also explain the principle of overlay networks and network plugins.

The Container Network Model

Docker has “networks”.

We can manage them with the `docker network` commands; for instance:

```
$ docker network ls
```

NETWORK ID	NAME	DRIVER
6bde79dfcf70	bridge	bridge
8d9c78725538	none	null
eb0eeab782f4	host	host
4c1ff84d6d3f	blog-dev	overlay
228a4355d548	blog-prod	overlay

New networks can be created (with `docker network create`).

(Note: networks `none` and `host` are special; let's set them aside for now.)



What's a network?

- Conceptually, a Docker “network” is a virtual switch
(we can also think about it like a VLAN, or a WiFi SSID, for instance)
- By default, containers are connected to a single network
(but they can be connected to zero, or many networks, even dynamically)
- Each network has its own subnet (IP address range)
- A network can be local (to a single Docker Engine) or global (span multiple hosts)
- Containers can have *network aliases* providing DNS-based service discovery
(and each network has its own “domain”, “zone”, or “scope”)

- A container can be given a network alias
(e.g. with `docker run --net some-network --net-alias db ...`)
- The containers running in the same network can resolve that network alias
(i.e. if they do a DNS lookup on `db`, it will give the container's address)
- We can have a different `db` container in each network
(this avoids naming conflicts between different stacks)
- When we name a container, it automatically adds the name as a network alias
(i.e. `docker run --name xyz ...` is like `docker run --net-alias xyz ...`)

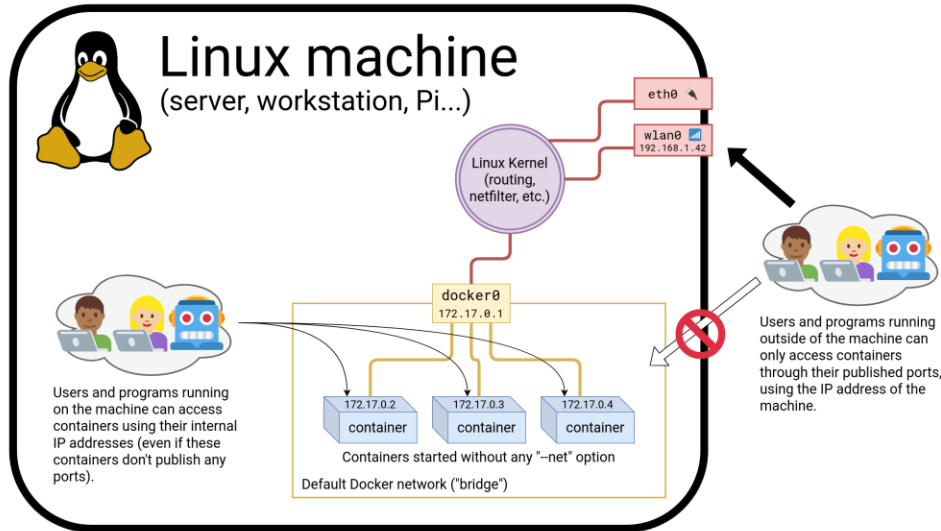
Network isolation

- Networks are isolated
- By default, containers in network A cannot reach those in network B
- A container connected to both networks A and B can act as a router or proxy
- Published ports are always reachable through the Docker host address
(`docker run -P . . .` makes a container port available to everyone)

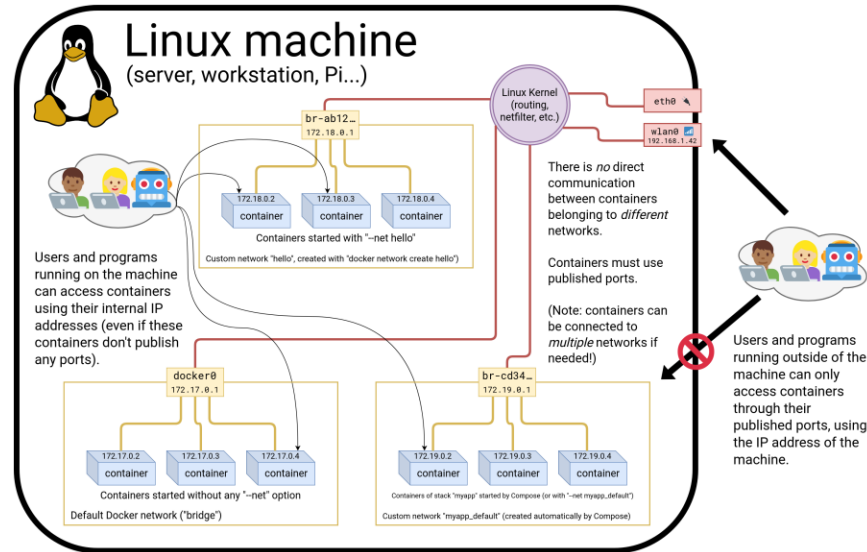
How to use networks

- We typically create one network per “stack” or app that we deploy
- More complex apps or stacks might require multiple networks
(e.g. `frontend`, `backend`, ...)
- Networks allow us to deploy multiple copies of the same stack
(e.g. `prod`, `dev`, `pr-442`, ...)
- If we use Docker Compose, this is managed automatically for us

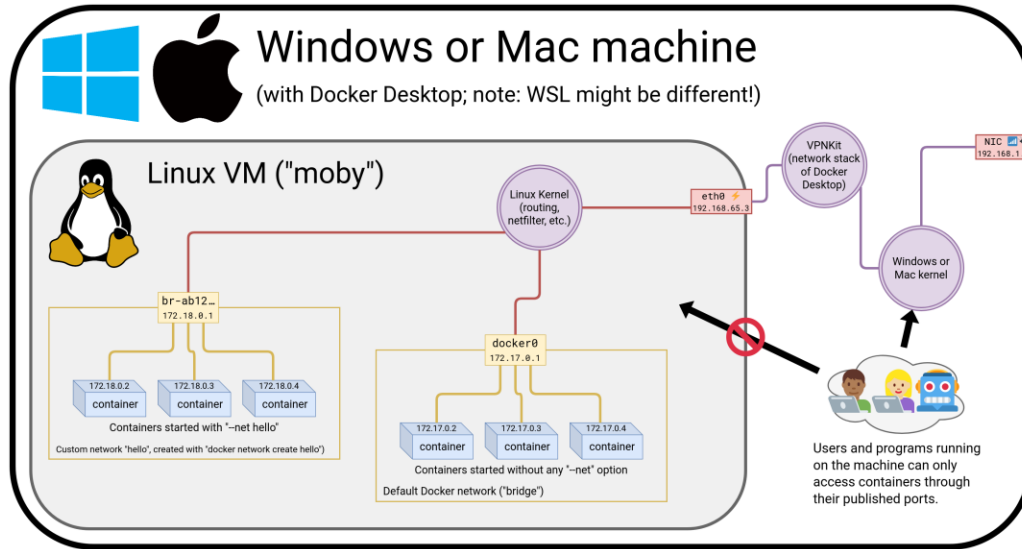




Multiple containers on the default bridge network, on a Linux machine



Multiple containers in multiple bridge networks, on a Linux machine



Multiple containers in multiple bridge networks, on a Mac/Windows machine

- CNM is the model used by Docker
- Kubernetes uses a different model, architected around CNI
(CNI is a kind of API between a container engine and *CNI plugins*)
- Docker model:
 - multiple isolated networks
 - per-network service discovery
 - network interconnection requires extra steps
- Kubernetes model:
 - single flat network
 - per-namespace service discovery
 - network isolation requires extra steps (Network Policies)

Creating a network

Let's create a network called dev.

```
$ docker network create dev
4c1ff84d6d3f1733d3e233ee039cac276f425a9d5228a4355d5
4878293a889ba
```

The network is now visible with the `network ls` command:

```
$ docker network ls
```

NETWORK ID	NAME	DRIVER
6bde79dfcf70	bridge	bridge
8d9c78725538	none	null
eb0eeab782f4	host	host
4c1ff84d6d3f	dev	bridge

Placing containers on a network

We will create a *named* container on this network.

It will be reachable with its name, `es`.

```
$ docker run -d --name es --net dev elasticsearch:2  
8abb80e229ce8926c7223beb69699f5f34d6f1d438bfc5682db  
893e798046863
```



Communication between containers

Now, create another container on this network.

```
$ docker run -ti --net dev alpine sh  
root@0ecccdfa45ef:/#
```

From this new container, we can resolve and ping the other one, using its assigned name:

```
/ # ping es  
PING es (172.18.0.2) 56(84) bytes of data.  
64 bytes from es.dev (172.18.0.2): icmp_seq=1 ttl=64 time=0.221 ms  
64 bytes from es.dev (172.18.0.2): icmp_seq=2 ttl=64 time=0.114 ms  
64 bytes from es.dev (172.18.0.2): icmp_seq=3 ttl=64 time=0.114 ms  
^C  
--- es ping statistics ---  
3 packets transmitted, 3 received, 0% packet loss, time 2000ms  
rtt min/avg/max/mdev = 0.114/0.149/0.221/0.052 ms  
root@0ecccdfa45ef:/#
```



Resolving container addresses

Since Docker Engine 1.10, name resolution is implemented by a dynamic resolver.

Archeological note: when CNM was introduced (in Docker Engine 1.9, November 2015) name resolution was implemented with `/etc/hosts`, and it was updated each time **CONTAINERS** were added/removed. This could cause interesting race conditions since `/etc/hosts` was a bind-mount (and couldn't be updated atomically).

```
[root@0ecccdfa45ef /]# cat /etc/hosts
172.18.0.3    0ecccdfa45ef
127.0.0.1     localhost
::1          localhost ip6-localhost ip6-loopback
fe00::0      ip6-localnet
ff00::0      ip6-mcastprefix
ff02::1      ip6-allnodes
ff02::2      ip6-allrouters
172.18.0.2    es
172.18.0.2    es.dev
```





Service discovery with containers

- Let's try to run an application that requires two containers.
- The first container is a web server.
- The other one is a redis data store.
- We will place them both on the `dev` network created before.



Running the web server

- The application is provided by the container image `jpetazzo/trainingwheels`.
- We don't know much about it so we will try to run it and see what happens!

Start the container, exposing all its ports:

```
$ docker run --net dev -d -P  
jpetazzo/trainingwheels
```

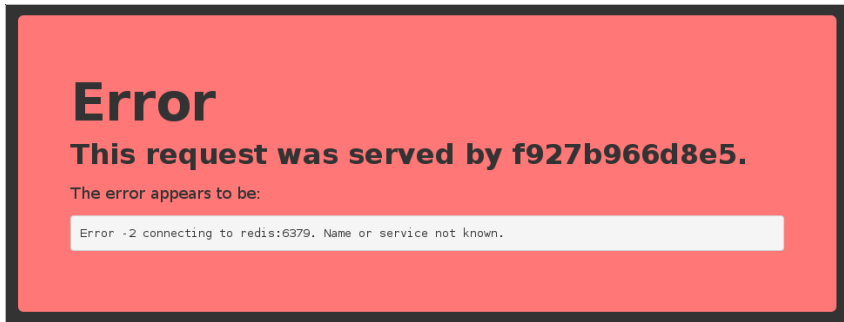
Check the port that has been allocated to it:

```
$ docker ps -l
```

Test the web server

- If we connect to the application now, we will see an error page:
- This is because the Redis service is not running.
- This container tries to resolve the name `redis`.

Note: we're not using a FQDN or an IP address here; just `redis`.



Trainingwheels error

Start the data store

- We need to start a Redis container.
- That container must be on the same network as the web server.
- It must have the right network alias (`redis`) so the application can find it.

Start the container:

```
$ docker run --net dev --net-alias redis -d redis
```



Test the web server again

- If we connect to the application now, we should see that the app is working correctly:
- When the app tries to resolve `redis`, instead of getting a DNS error, it gets the IP address of our Redis container.

Training wheels

This request was served by f927b966d8e5.

f927b966d8e5 served 1 request so far.

The current ladder is:

- f927b966d8e5 → 1 request

Trainingwheels OK

A few words on scope

- Container names are unique (there can be only one `--name redis`)
- Network aliases are not unique
- We can have the same network alias in different networks:

```
docker run --net dev --net-alias redis ...  
docker run --net prod --net-alias redis ...
```
- We can even have multiple containers with the same alias in the same network
(in that case, we get multiple DNS entries, aka “DNS round robin”)

Names are *local* to each network

Let's try to ping our `es` container from another container, when that other container is *not* on the `dev` network.

```
$ docker run --rm alpine ping es  
ping: bad address 'es'
```

Names can be resolved only when containers are on the same network.

Containers can contact each other only when they are on the same network (you can try to ping using the IP address to verify).



Network aliases

We would like to have another network, `prod`, with its own `es` container. But there can be only one container named `es`!

We will use *network aliases*.

A container can have multiple network aliases.

Network aliases are *local* to a given network (only exist in this network).

Multiple containers can have the same network alias (even on the same network).

Since Docker Engine 1.11, resolving a network alias yields the IP addresses of all containers holding this alias.



Creating containers on another network

Create the `prod` network.

```
$ docker network create prod
5a41562fecf2d8f115bedc16865f7336232a04268bdf2bd816aecca01b68
d50c
```

We can now create multiple containers with the `es` alias on the new `prod` network.

```
$ docker run -d --name prod-es-1 --net-alias es --net prod
elasticsearch:2
38079d21caf0c5533a391700d9e9e920724e89200083df73211081c8a356
d771
$ docker run -d --name prod-es-2 --net-alias es --net prod
elasticsearch:2
1820087a9c600f43159688050dcc164c298183e1d2e62d5694fd46b10ac3
bc3d
```



Resolving network aliases

Let's try DNS resolution first, using the `nslookup` tool that ships with the `alpine` image.

```
$ docker run --net prod --rm alpine nslookup es
```

```
Name:          es
```

```
Address 1: 172.23.0.3 prod-es-2.prod
```

```
Address 2: 172.23.0.2 prod-es-1.prod
```

(You can ignore the `can't resolve ' (null) ' errors.`)



Connecting to aliased containers

Each Elasticsearch instance has a name (generated when it is started). This name can be seen when we issue a simple HTTP request on the Elasticsearch API endpoint.

Try the following command a few times:

```
$ docker run --rm --net dev centos curl -s es:9200
{
  "name" : "Tarot",
  ...
}
```

Then try it a few times by replacing `--net dev` with `--net prod`:

```
$ docker run --rm --net prod centos curl -s es:9200
{
  "name" : "The Symbiote",
  ...
}
```


- Docker will not create network names and aliases on the default `bridge` network.
- Therefore, if you want to use those features, you have to create a custom network first.
- Network aliases are *not* unique on a given network.
- i.e., multiple containers can have the same alias on the same network.
- In that scenario, the Docker DNS server will return multiple records. (i.e. you will get DNS round robin out of the box.)
- Enabling *Swarm Mode* gives access to clustering and load balancing with IPVS.
- Creation of networks and network aliases is generally automated with tools like Compose.

A few words about round robin DNS

Don't rely exclusively on round robin DNS to achieve load balancing.

Many factors can affect DNS resolution, and you might see:

- all traffic going to a single instance;
- traffic being split (unevenly) between some instances;
- different behavior depending on your application language;
- different behavior depending on your base distro;
- different behavior depending on other factors (sic).

It's OK to use DNS to discover available endpoints, but remember that you have to re-resolve every now and then to discover new endpoints.



When creating a network, extra options can be provided.

- `--internal` disables outbound traffic (the network won't have a default gateway).
- `--gateway` indicates which address to use for the gateway (when outbound traffic is allowed).
- `--subnet` (in CIDR notation) indicates the subnet to use.
- `--ip-range` (in CIDR notation) indicates the subnet to allocate from.
- `--aux-address` allows specifying a list of reserved addresses (which won't be allocated to containers).

Setting containers' IP address

- It is possible to set a container's address with `--ip`.
- The IP address has to be within the subnet used for the container.

A full example would look like this.

```
$ docker network create --subnet 10.66.0.0/16 pubnet
42fb16ec412383db6289a3e39c3c0224f395d7f85bcb1859b279e7
a564d4e135
$ docker run --net pubnet --ip 10.66.66.66 -d nginx
b2887adeb5578a01fd9c55c435cad56bbbe802350711d2743691f9
5743680b09
```

Note: don't hard code container IP addresses in your code!

I repeat: don't hard code container IP addresses in your code!



- A network is managed by a *driver*.
- The built-in drivers include:
 - `bridge` (default)
 - `none`
 - `host`
 - `macvlan`
 - `overlay` (for Swarm clusters)
- More drivers can be provided by plugins (OVS, VLAN...)
- A network can have a custom IPAM (IP allocator).

Overlay networks

- The features we've seen so far only work when all containers are on a single host.
- If containers span multiple hosts, we need an *overlay* network to connect them together.
- Docker ships with a default network plugin, `overlay`, implementing an overlay network leveraging VXLAN, *enabled with Swarm Mode*.
- Other plugins (Weave, Calico...) can provide overlay networks as well.
- Once you have an overlay network, *all the features that we've used in this chapter work identically across multiple hosts*.



Multi-host networking (overlay)

Out of the scope for this intro-level workshop!

Very short instructions:

- enable Swarm Mode (`docker swarm init` then `docker swarm join` on other nodes)
- `docker network create mynet --driver overlay`
- `docker service create --network mynet myimage`

If you want to learn more about Swarm mode, you can check [this video](#) or [these slides](#).



Out of the scope for this intro-level workshop!

General idea:

- install the plugin (they often ship within containers)
- run the plugin (if it's in a container, it will often require extra parameters; don't just `docker run` it blindly!)
- some plugins require configuration or activation (creating a special file that tells Docker “use the plugin whose control socket is at the following location”)
- you can then `docker network create --driver pluginname`

Connecting and disconnecting dynamically

- So far, we have specified which network to use when starting the container.
- The Docker Engine also allows connecting and disconnecting while the container is running.
- This feature is exposed through the Docker API, and through two Docker CLI commands:
 - `docker network connect <network> <container>`
 - `docker network disconnect <network> <container>`



Dynamically connecting to a network

- We have a container named `es` connected to a network named `dev`.
- Let's start a simple alpine container on the default network:

```
$ docker run -ti alpine sh  
/ #
```

- In this container, try to ping the `es` container:

```
/ # ping es  
ping: bad address 'es'
```

This doesn't work, but we will change that by connecting the container.



Finding the container ID and connecting it

- Figure out the ID of our alpine container; here are two methods:
 - looking at `/etc/hostname` in the container,
 - running `docker ps -lq` on the host.
- Run the following command on the host:

```
$ docker network connect dev <container_id>
```

Checking what we did

- Try again to ping es from the container.
- It should now work correctly:

```
/ # ping es
```

```
PING es (172.20.0.3): 56 data bytes
```

```
64 bytes from 172.20.0.3: seq=0 ttl=64 time=0.376 ms
```

```
64 bytes from 172.20.0.3: seq=1 ttl=64 time=0.130 ms
```

```
^C
```

- Interrupt it with Ctrl-C.



Looking at the network setup in the container

We can look at the list of network interfaces with `ifconfig`, `ip a`, or `ip l`:

```
/ # ip a
1: lo: <LOOPBACK,UP,LOWER_UP> mtu 65536 qdisc noqueue state UNKNOWN qlen 1000
    link/loopback 00:00:00:00:00:00 brd 00:00:00:00:00:00
    inet 127.0.0.1/8 scope host lo
        valid_lft forever preferred_lft forever
18: eth0@if19: <BROADCAST,MULTICAST,UP,LOWER_UP,M-DOWN> mtu 1500 qdisc noqueue state UP
    link/ether 02:42:ac:11:00:02 brd ff:ff:ff:ff:ff:ff
    inet 172.17.0.2/16 brd 172.17.255.255 scope global eth0
        valid_lft forever preferred_lft forever
20: eth1@if21: <BROADCAST,MULTICAST,UP,LOWER_UP,M-DOWN> mtu 1500 qdisc noqueue state UP
    link/ether 02:42:ac:14:00:04 brd ff:ff:ff:ff:ff:ff
    inet 172.20.0.4/16 brd 172.20.255.255 scope global eth1
        valid_lft forever preferred_lft forever
/ #
```

Each network connection is materialized with a virtual network interface.

As we can see, we can be connected to multiple networks at the same time.

Disconnecting from a network

- Let's try the symmetrical command to disconnect the container:

```
$ docker network disconnect dev <container_id>
```

- From now on, if we try to ping `es`, it will not resolve:

```
/ # ping es  
ping: bad address 'es'
```

- Trying to ping the IP address directly won't work either:

```
/ # ping 172.20.0.3  
... (nothing happens until we interrupt it with Ctrl-C)
```



Network aliases are scoped per network

- Each network has its own set of network aliases.
- We saw this earlier: `es` resolves to different addresses in `dev` and `prod`.
- If we are connected to multiple networks, the resolver looks up names in each of them (as of Docker Engine 18.03, it is the connection order) and stops as soon as the name is found.
- Therefore, if we are connected to both `dev` and `prod`, resolving `es` will **not** give us the addresses of all the `es` services; but only the ones in `dev` or `prod`.
- However, we can lookup `es.dev` or `es.prod` if we need to.



Finding out about our networks and names

- We can do reverse DNS lookups on containers' IP addresses.
- If the IP address belongs to a network (other than the default bridge), the result will be:

```
name-or-first-alias-or-container-id.network-name
```

- Example:

```
$ docker run -ti --net prod --net-alias hello alpine
/ # apk add --no-cache drill
...
OK: 5 MiB in 13 packages
/ # ifconfig
eth0      Link encap:Ethernet  HWaddr 02:42:AC:15:00:03
          inet addr: `172.21.0.3`  Bcast:172.21.255.255  Mask:255.255.0.0
          UP BROADCAST RUNNING MULTICAST  MTU:1500  Metric:1
...
/ # drill -t ptr `3.0.21.172`.in-addr.arpa
...
;; ANSWER SECTION:
3.0.21.172.in-addr.arpa.      600 IN    PTR `hello.prod`.
...
```



Building with a custom network

- We can build a Dockerfile with a custom network with `docker build --network NAME`.
- This can be used to check that a build doesn't access the network.
(But keep in mind that most Dockerfiles will fail, because they need to install remote packages and dependencies!)
- This may be used to access an internal package repository.
(But try to use a multi-stage build instead, if possible!)



- YAML Ain't Markup Language (according to yaml.org)
- *Almost* required when working with containers:
 - Docker Compose files
 - Kubernetes manifests
 - Many CI pipelines (GitHub, GitLab...)
- If you don't know much about YAML, this is for you!



What is it?

- Data representation language

- `country: France`
`capital: Paris`
`code: fr`
`population: 68042591`
- `country: Germany`
`capital: Berlin`
`code: de`
`population: 84270625`
- `country: Norway`
`capital: Oslo`
`code: no # It's a trap!`
`population: 5425270`

- Even without knowing YAML, we probably can add a country to that file :)



Trying YAML

- Method 1: in the browser

<https://onlineyamltools.com/convert-yaml-to-json>

<https://onlineyamltools.com/highlight-yaml>

- Method 2: in a shell

```
yq . foo.yaml
```

- Method 3: in Python

```
import yaml; yaml.safe_load("""  
- country: France  
  capital: Paris  
""")
```

Basic stuff

- Strings, numbers, boolean values, `null`
- Sequences (=arrays, lists)
- Mappings (=objects)
- Superset of JSON
(if you know JSON, you can just write JSON)
- Comments start with `#`
- A single *file* can have multiple *documents*
(separated by `---` on a single line)



Sequences

- Example: sequence of strings

```
[ "france", "germany", "norway" ]
```

- Example: the same sequence, without the double-quotes

```
[ france, germany, norway ]
```

- Example: the same sequence, in “block collection style” (=multi-line)

```
- france  
- germany  
- norway
```

- Example: mapping strings to numbers

```
{ "france": 68042591, "germany": 84270625, "norway":  
5425270 }
```

- Example: the same mapping, without the double-quotes

```
{ france: 68042591, germany: 84270625, norway: 5425270 }
```

- Example: the same mapping, in “block collection style”

```
france: 68042591  
germany: 84270625  
norway: 5425270
```

Combining types

- In a sequence (or mapping) we can have different types (including other sequences or mappings)

- Example:

```
questions: [ name, quest, favorite color ]
answers: [ "Arthur, King of the Britons", Holy Grail,
purple, 42 ]
```

- Note that we need to quote “Arthur” because of the comma
- Note that we don’t have the same number of elements in questions and answers

More combinations

- Example:

- service: nginx
ports: [80, 443]
 - service: bind
ports: [53/tcp, 53/udp]
 - service: ssh
ports: 22

- Note that `ports` doesn't always have the same type
(the code handling that data will probably have to be smart!)



Automatic booleans

```
codes:  
  france: fr  
  germany: de  
  norway: no
```

—

```
{  
  "codes": {  
    "france": "fr",  
    "germany": "de",  
    "norway": false  
  }  
}
```



Automatic booleans

- `no` can become `false`
(it depends on the YAML parser used)
- It should be quoted instead:

```
codes:  
  france: fr  
  germany: de  
  norway: "no"
```



Automatic floats

```
version:  
  libfoo: 1.10  
  foctl: 1.0
```

—

```
{  
  "version": {  
    "libfoo": 1.1,  
    "foctl": 1  
  }  
}
```



Automatic floats

- Trailing zeros disappear
- These should also be quoted:

```
version:  
  libfoo: "1.10"  
  foctl: "1.0"
```



Automatic times

```
portmap:
```

```
- 80:80
```

```
- 22:22
```

```
-
```

```
{
```

```
  "portmap": [
```

```
    "80:80",
```

```
    1342
```

```
  ]
```

```
}
```



Automatic times

- 22 : 22 becomes 1342
- That's 22 minutes and 22 seconds = 1342 seconds
- Again, it should be quoted

Document separator

- A single YAML *file* can have multiple *documents* separated by `---`:

```
This is a document consisting of a single string.
```

```
---
```

```
name: The second document
```

```
type: This one is a mapping (key→value)
```

```
---
```

```
- Third document
```

```
- This one is a sequence
```

- Some folks like to add an extra `---` at the beginning and/or at the end (it's not mandatory but can help e.g. to `cat` multiple files together)
- .footnote[💡 Ignore this; it's here to work around [this issue](#).]

Multi-line strings

Try the following block in a YAML parser:

```
add line breaks: "in double quoted strings\n(like this)"
preserve line break: |
    by using
    a pipe (|)
    (this is great for embedding shell scripts,
    configuration files...)
do not preserve line breaks: >
    by using
    a greater-than (>)
    (this is great for embedding very long lines)
```

See <https://yaml-multiline.info/> for advanced multi-line tips!

(E.g. to strip or keep extra `\n` characters at the end of the block.)



Anchors let you “memorize” and re-use content:

```
debian: &debian  
  packages: deb  
  latest-stable: bullseye
```

```
also-debian: *debian
```

```
ubuntu:  
  <<: *debian  
  latest-stable: jammy
```

YAML, good or evil?

- Natural progression from XML to JSON to YAML
- There are other data languages out there
(e.g. HCL, domain-specific things crafted with Ruby, CUE...)
- Compromises are made, for instance:
 - more user-friendly → more “magic” with side effects
 - more powerful → steeper learning curve
- Love it or loathe it but it’s a good idea to understand it!
- Interesting tool if you appreciate YAML: <https://carvel.dev/ytt/>



Dockerfile = great to build *one* container image.

What if we have multiple containers?

What if some of them require particular `docker run` parameters?

How do we connect them all together?

... Compose solves these use-cases (and a few more).



Before we had Compose, we would typically write custom scripts to:

- build container images,
- run containers using these images,
- connect the containers together,
- rebuild, restart, update these images and containers.

Compose enables a simple, powerful onboarding workflow:

1. Checkout our code.
2. Run `docker-compose up`.
3. Our app is up and running!



composeup

(Or: when do we need something else?)

- Compose is *not* an orchestrator
- It isn't designed to need to run containers on multiple nodes
(it can, however, work with Docker Swarm Mode)
- Compose isn't ideal if we want to run containers on Kubernetes
 - it uses different concepts (Compose services $\neq$ Kubernetes services)
 - it needs a Docker Engine (although containerd support might be coming)

First rodeo with Compose

1. Write Dockerfiles
2. Describe our stack of containers in a YAML file called `docker-compose.yml`
3. `docker-compose up` (or `docker-compose up -d` to run in the background)
4. Compose pulls and builds the required images, and starts the containers
5. Compose shows the combined logs of all the containers
(if running in the background, use `docker-compose logs`)
6. Hit Ctrl-C to stop the whole stack
(if running in the background, use `docker-compose stop`)



After making changes to our source code, we can:

1. `docker-compose build` to rebuild container images
2. `docker-compose up` to restart the stack with the new images

We can also combine both with `docker-compose up --build`

Compose will be smart, and only recreate the containers that have changed.

When working with interpreted languages:

- don't rebuild each time
- leverage a `volumes` section instead

Launching Our First Stack with Compose

First step: clone the source code for the app we will be working on.

```
git clone  
https://github.com/jpetazzo/trainingwheels  
cd trainingwheels
```

Second step: start the app.

```
docker-compose up
```

Watch Compose build and run the app.

That Compose stack exposes a web server on port 8000; try connecting to it.



We should see a web page like this:

Each time we reload, the counter should increase.

Training wheels

This request was served by 5457fb09c174.

5457fb09c174 served 1 request so far.

The current ladder is:

- 5457fb09c174 → 1 request

composeapp

Stopping the app

When we hit Ctrl-C, Compose tries to gracefully terminate all of the containers.

After ten seconds (or if we press `^C` again) it will forcibly kill them.



The docker-compose.yml file

Here is the file used in the demo:

```
version: "3"

services:
  www:
    build: www
    ports:
      - ${PORT-8000}:5000
    user: nobody
    environment:
      DEBUG: 1
    command: python counter.py
    volumes:
      - ./www:/src

  redis:
    image: redis
```



A Compose file has multiple sections:

- `version` is mandatory. (Typically use “3”.)
- `services` is mandatory. Each service corresponds to a container.
- `networks` is optional and indicates to which networks containers should be connected. (By default, containers will be connected on a private, per-compose-file network.)
- `volumes` is optional and can define volumes to be used and/or shared by the containers.

Compose file versions

- Version 1 is legacy and shouldn't be used.
(If you see a Compose file without `version` and `services`, it's a legacy v1 file.)
- Version 2 added support for networks and volumes.
- Version 3 added support for deployment options (scaling, rolling updates, etc).
- Typically use `version: "3"`.

The [Docker documentation](#) has excellent information about the Compose file format if you need to know more about versions.



Each service in the YAML file must contain either `build`, or `image`.

- `build` indicates a path containing a Dockerfile.
- `image` indicates an image name (local, or on a registry).
- If both are specified, an image will be built from the `build` directory and named `image`.

The other parameters are optional.

They encode the parameters that you would typically add to `docker run`.

Sometimes they have several minor improvements.

Container parameters

- `command` indicates what to run (like `CMD` in a Dockerfile).
- `ports` translates to one (or multiple) `-p` options to map ports. You can specify local ports (i.e. `x:y` to expose public port `x`).
- `volumes` translates to one (or multiple) `-v` options. You can use relative paths here.

For the full list, check: <https://docs.docker.com/compose/compose-file/>



Environment variables

- We can use environment variables in Compose files
(like `$THIS` or `${THAT}`)
- We can provide default values, e.g. `${PORT-8000}`
- Compose will also automatically load the environment file `.env`
(it should contain `VAR=value`, one per line)
- This is a great way to customize build and run parameters
(base image versions to use, build and run secrets, port numbers...)



Configuring a Compose stack

- Follow [12-factor app configuration principles](#)
(configure the app through environment variables)
- Provide (in the repo) a default environment file suitable for development
(no secret or sensitive value)
- Copy the default environment file to `.env` and tweak it
(or: provide a script to generate `.env` from a template)



Running multiple copies of a stack

- Copy the stack in two different directories, e.g. `front` and `frontcopy`
- Compose prefixes images and containers with the directory name:
`front_www, front_www_1, front_db_1`
`frontcopy_www, frontcopy_www_1, frontcopy_db_1`
- Alternatively, use `docker-compose -p frontcopy`
(to set the `--project-name` of a stack, which default to the dir name)
- Each copy is isolated from the others (runs on a different network)

Checking stack status

We have `ps`, `docker ps`, and similarly, `docker-compose ps`:

```
$ docker-compose ps
```

Name	Command	State
Ports		

trainingwheels_redis_1	/entrypoint.sh red	Up
6379/tcp		
trainingwheels_www_1	python counter.py	Up
0.0.0.0:8000->5000/tcp		

Shows the status of all the containers of our stack.

Doesn't show the other containers.



Cleaning up (1)

If you have started your application in the background with Compose and want to stop it easily, you can use the `kill` command:

```
$ docker-compose kill
```

Likewise, `docker-compose rm` will let you remove containers (after confirmation):

```
$ docker-compose rm
```

```
Going to remove trainingwheels_redis_1,  
trainingwheels_www_1
```

```
Are you sure? [yN] y
```

```
Removing trainingwheels_redis_1...
```

```
Removing trainingwheels_www_1...
```



Cleaning up (2)

Alternatively, `docker-compose down` will stop and remove containers.

It will also remove other resources, like networks that were created for the application.

```
$ docker-compose down
Stopping trainingwheels_www_1 ... done
Stopping trainingwheels_redis_1 ... done
Removing trainingwheels_www_1 ... done
Removing trainingwheels_redis_1 ... done
```

Use `docker-compose down -v` to remove everything including volumes.

Special handling of volumes

- When an image gets updated, Compose automatically creates a new container
- The data in the old container is lost...
- ...Except if the container is using a *volume*
- Compose will then re-attach that volume to the new container
(and data is then retained across database upgrades)
- All good database images use volumes
(e.g. all official images)



Gotchas with volumes

- Unfortunately, Docker volumes don't have labels or metadata
- Compose tracks volumes thanks to their associated container
- If the container is deleted, the volume gets orphaned
- **Example:** `docker-compose down && docker-compose up`
 - the old volume still exists, detached from its container
 - a new volume gets created
- `docker-compose down -v/--volumes` **deletes volumes**
(but **not** `docker-compose down && docker-compose down -v`!)



Option 1: *named volumes*

```
services:
  app:
    volumes:
      - data:/some/path
volumes:
  data:
```

- Volume will be named `<project>_data`
- It won't be orphaned with `docker-compose down`
- It will correctly be removed with `docker-compose down -v`

Option 2: *relative paths*

```
services:  
  app:  
    volumes:  
      - ./data:/some/path
```

- Makes it easy to colocate the app and its data
(for migration, backups, disk usage accounting...)
- Won't be removed by `docker-compose down -v`

Managing complex stacks

- Compose provides multiple features to manage complex stacks (with many containers)
- `-f/--file/$COMPOSE_FILE` can be a list of Compose files (separated by `:` and merged together)
- Services can be assigned to one or more *profiles*
- `--profile/$COMPOSE_PROFILE` can be a list of comma-separated profiles
(see [Using service profiles](#) in the Compose documentation)
- These variables can be set in `.env`



Dependencies

- A service can have a `depends_on` section (listing one or more other services)
 - This is used when bringing up individual services (e.g. `docker-compose up blah` or `docker-compose run foo`)
-  It doesn't make a service “wait” for another one to be up!

A bit of history and trivia

- Compose was initially named “Fig”
- Compose is one of the only components of Docker written in Python (almost everything else is in Go)
- In 2020, Docker introduced “Compose CLI”:
 - `docker compose` command to deploy Compose stacks to some clouds
 - progressively getting feature parity with `docker-compose`
 - also provides numerous improvements (e.g. leverages BuildKit by default)



Let's write a Compose file for the wordsmith app!

The code is at: <https://github.com/jpetazzo/wordsmith> # Reducing image size



- In the previous example, our final image contained:
 - our `hello` program
 - its source code
 - the compiler
- Only the first one is strictly necessary.
- We are going to see how to obtain an image without the superfluous components.

Can't we remove superfluous files with RUN?

What happens if we do one of the following commands?

- `RUN rm -rf ...`
- `RUN apt-get remove ...`
- `RUN make clean ...`

–

This adds a layer which removes a bunch of files.

But the previous layers (which added the files) still exist.



Removing files with an extra layer

When downloading an image, all the layers must be downloaded.

Dockerfile instruction	Layer size	Image size
FROM ubuntu	Size of base image	Size of base image
...	...	Sum of this layer + all previous ones
RUN apt-get install somepackage	Size of files added (e.g. a few MB)	Sum of this layer + all previous ones
...	...	Sum of this layer + all previous ones
RUN apt-get remove somepackage	Almost zero (just metadata)	Same as previous one

Therefore, `RUN rm` does not reduce the size of the image or free up disk space.



Various techniques are available to obtain smaller images:

- collapsing layers,
- adding binaries that are built outside of the Dockerfile,
- squashing the final image,
- multi-stage builds.

Let's review them quickly.

Collapsing layers

You will frequently see Dockerfiles like this:

```
FROM ubuntu
RUN apt-get update && apt-get install xxx && ... && apt-get
remove xxx && ...
```

Or the (more readable) variant:

```
FROM ubuntu
RUN apt-get update \
    && apt-get install xxx \
    && ... \
    && apt-get remove xxx \
    && ...
```

This RUN command gives us a single layer.

The files that are added, then removed in the same layer, do not grow the layer size.



Collapsing layers: pros and cons

Pros:

- works on all versions of Docker
- doesn't require extra tools

Cons:

- not very readable
- some unnecessary files might still remain if the cleanup is not thorough
- that layer is expensive (slow to build)

Building binaries outside of the Dockerfile

This results in a Dockerfile looking like this:

```
FROM ubuntu  
COPY xxx /usr/local/bin
```

Of course, this implies that the file `xxx` exists in the build context.

That file has to exist before you can run `docker build`.

For instance, it can:

- exist in the code repository,
- be created by another tool (script, Makefile...),
- be created by another container image and extracted from the image.

See for instance the [busybox official image](#) or this [older busybox image](#).



Building binaries outside: pros and cons

Pros:

- final image can be very small

Cons:

- requires an extra build tool
- we're back in dependency hell and “works on my machine”

Cons, if binary is added to code repository:

- breaks portability across different platforms
- grows repository size a lot if the binary is updated frequently



Squashing the final image

The idea is to transform the final image into a single-layer image.

This can be done in (at least) two ways.

- Activate experimental features and squash the final image:

```
docker image build --squash ...
```

- Export/import the final image.

```
docker build -t temp-image .
```

```
docker run --entrypoint true --name temp-container temp-image
```

```
docker export temp-container | docker import - final-image
```

```
docker rm temp-container
```

```
docker rmi temp-image
```



Squashing the image: pros and cons

Pros:

- single-layer images are smaller and faster to download
- removed files no longer take up storage and network resources

Cons:

- we still need to actively remove unnecessary files
- squash operation can take a lot of time (on big images)
- squash operation does not benefit from cache (even if we change just a tiny file, the whole image needs to be re-squashed)

Multi-stage builds

Multi-stage builds allow us to have multiple *stages*.
Each stage is a separate image, and can copy files from previous stages.
We're going to see how they work in more detail.



Multi-stage builds



- At any point in our `Dockerfile`, we can add a new `FROM` line.
- This line starts a new stage of our build.
- Each stage can access the files of the previous stages with `COPY --from=...`
- When a build is tagged (with `docker build -t ...`), the last stage is tagged.
- Previous stages are not discarded: they will be used for caching, and can be referenced.

Multi-stage builds in practice

- Each stage is numbered, starting at 0
- We can copy a file from a previous stage by indicating its number, e.g.:

```
COPY --from=0 /file/from/first/stage  
/location/in/current/stage
```

- We can also name stages, and reference these names:

```
FROM golang AS builder  
RUN ...  
FROM alpine  
COPY --from=builder /go/bin/mylittlebinary /usr/local/bin/
```



Multi-stage builds for our C program

We will change our Dockerfile to:

- give a nickname to the first stage: `compiler`
- add a second stage using the same `ubuntu` base image
- add the `hello` binary to the second stage
- make sure that `CMD` is in the second stage

The resulting Dockerfile is on the next slide.



Multi-stage build Dockerfile

Here is the final Dockerfile:

```
FROM ubuntu AS compiler
RUN apt-get update
RUN apt-get install -y build-essential
COPY hello.c /
RUN make hello
FROM ubuntu
COPY --from=compiler /hello /hello
CMD /hello
```

Let's build it, and check that it works correctly:

```
docker build -t hellomultistage .
docker run hellomultistage
```



Comparing single/multi-stage build image sizes

List our images with `docker images`, and check the size of:

- the `ubuntu` base image,
- the single-stage `hello` image,
- the multi-stage `hellomultistage` image.

We can achieve even smaller images if we use smaller base images.

However, if we use common base images (e.g. if we standardize on `ubuntu`), these common images will be pulled only once per node, so they are virtually “free.”



Build targets

- We can also tag an intermediary stage with the following command:
`docker build --target STAGE --tag NAME`
- This will create an image (named NAME) corresponding to stage STAGE
- This can be used to easily access an intermediary stage for inspection
(instead of parsing the output of `docker build` to find out the image ID)
- This can also be used to describe multiple images from a single Dockerfile
(instead of using multiple Dockerfiles, which could go out of sync)

—



Dealing with download caches

- In some cases, our images contain temporary downloaded files or caches
(examples: packages downloaded by `pip`, Maven, etc.)
- These can sometimes be disabled
(e.g. `pip install --no-cache-dir ...`)
- The cache can also be cleaned immediately after installing
(e.g. `pip install ... && rm -rf ~/.cache/pip`)



Download caches and multi-stage builds

- Download+install packages in a build stage
- Copy the installed packages to a run stage
- Example: in the specific case of Python, use a virtual env
(install in the virtual env; then copy the virtual env directory)



Download caches and BuildKit

- BuildKit has a caching feature for run stages
- It can address download caches elegantly
- Example:

```
RUN --mount=type=cache,target=/pipcache pip install --  
cache-dir /pipcache ...
```

- The cache won't be in the final image, but it'll persist across builds

We have built our first images.

We can now publish it to the Docker Hub!

You don't have to do the exercises in this section, because they require an account on the Docker Hub, and we don't want to force anyone to create one.

Note, however, that creating an account on the Docker Hub is free (and doesn't require a credit card), and hosting public images is free as well.



Logging into our Docker Hub account

- This can be done from the Docker CLI:

```
docker login
```

.warning[When running Docker for Mac/Windows, or Docker on a Linux workstation, it can (and will when possible) integrate with your system's keyring to store your credentials securely. However, on most Linux servers, it will store your credentials in `~/.docker/config.`]



Image tags and registry addresses

- Docker images tags are like Git tags and branches.
- They are like *bookmarks* pointing at a specific image ID.
- Tagging an image doesn't *rename* an image: it adds another tag.
- When pushing an image to a registry, the registry address is in the tag.

Example: `registry.example.net:5000/image`

- What about Docker Hub images?

—

- `jpetazzo/clock` is, in fact,
`index.docker.io/jpetazzo/clock`
- `ubuntu` is, in fact, `library/ubuntu`,
i.e. `index.docker.io/library/ubuntu`



Tagging an image to push it on the Hub

- Let's tag our `figlet` image (or any other to our liking):

```
docker tag figlet jpetazzo/figlet
```

- And push it to the Hub:

```
docker push jpetazzo/figlet
```

- That's it!

–

- Anybody can now `docker run jpetazzo/figlet` anywhere.



The goodness of automated builds

- You can link a Docker Hub repository with a GitHub or BitBucket repository
- Each push to GitHub or BitBucket will trigger a build on Docker Hub
- If the build succeeds, the new image is available on Docker Hub
- You can map tags and branches between source and container images
- If you work with public repositories, this is free



Setting up an automated build

- We need a Dockerized repository!
- Let's go to <https://github.com/jpetazzo/trainingwheels> and fork it.
- Go to the Docker Hub (<https://hub.docker.com/>) and sign-in. Select “Repositories” in the blue navigation menu.
- Select “Create” in the top-right bar, and select “Create Repository+”.
- Connect your Docker Hub account to your GitHub account.
- Click “Create” button.
- Then go to “Builds” folder.
- Click on Github icon and select your user and the repository that we just forked.
- In “Build rules” block near page bottom, put `/www` in “Build Context” column (or whichever directory the Dockerfile is in).
- Click “Save and Build” to build the repository immediately (without waiting for a git push).
- Subsequent builds will happen automatically, thanks to GitHub hooks.



- Some services can build images on the fly from a repository

- Example: ctr.run

.lab[

- Use `ctr.run` to automatically build a container image and run it:

```
docker run ctr.run/github.com/undefinedlabs/hello-world
```

]

There might be a long pause before the first layer is pulled, because the API behind `docker pull` doesn't allow to stream build logs, and there is no feedback during the build.

It is possible to view the build logs by setting up an account on ctr.run.

We will see how to:

- Reduce the number of layers.
- Leverage the build cache so that builds can be faster.
- Embed unit testing in the build process.



Reducing the number of layers

- Each line in a `Dockerfile` creates a new layer.
- Build your `Dockerfile` to take advantage of Docker's caching system.
- Combine commands by using `&&` to continue commands and `\` to wrap lines.

Note: it is frequent to build a `Dockerfile` line by line:

```
RUN apt-get install thisthing
```

```
RUN apt-get install andthatthing andthatotherone
```

```
RUN apt-get install somemorestuff
```

And then refactor it trivially before shipping:

```
RUN apt-get install thisthing andthatthing  
andthatotherone somemorestuff
```

Avoid re-installing dependencies at each build

- Classic Dockerfile problem:
“each time I change a line of code, all my dependencies are re-installed!”
- **Solution:** COPY dependency lists (`package.json`, `requirements.txt`, etc.) by themselves to avoid reinstalling unchanged dependencies every time.



Example “bad” Dockerfile

The dependencies are reinstalled every time, because the build system does not know if `requirements.txt` has been updated.

```
FROM python
WORKDIR /src
COPY . .
RUN pip install -qr requirements.txt
EXPOSE 5000
CMD ["python", "app.py"]
```



Adding the dependencies as a separate step means that Docker can cache more efficiently and only install them when `requirements.txt` changes.

```
FROM python
WORKDIR /src
COPY requirements.txt .
RUN pip install -qr requirements.txt
COPY . .
EXPOSE 5000
CMD ["python", "app.py"]
```

Be careful with `chown`, `chmod`, `mv`

- Layers cannot store efficiently changes in permissions or ownership.
- Layers cannot represent efficiently when a file is moved either.
- As a result, operations like `chown`, `chmod`, `mv` can be expensive.
- For instance, in the Dockerfile snippet below, each `RUN` line creates a layer with an entire copy of `some-file`.

```
COPY some-file .  
RUN chown www-data:www-data some-file  
RUN chmod 644 some-file  
RUN mv some-file /var/www
```

- How can we avoid that?



Put files on the right place

- Instead of using `mv`, directly put files at the right place.
- When extracting archives (tar, zip...), merge operations in a single layer.

Example:

```
...  
RUN wget http://.../foo.tar.gz \  
  && tar -zxvf foo.tar.gz \  
  && mv foo/fooctl /usr/local/bin \  
  && rm -rf foo foo.tar.gz
```

...

- The Dockerfile instruction COPY can take a --chown parameter.

Examples:

...

```
COPY --chown=1000 some-file .
```

```
COPY --chown=1000:1000 some-file .
```

```
COPY --chown=www-data:www-data some-file .
```

- The --chown flag can specify a user, or a user:group pair.
- The user and group can be specified as names or numbers.
- When using names, the names must exist in /etc/passwd or /etc/group.

(In the container, not on the host!)

Set correct permissions locally

- Instead of using `chmod`, set the right file permissions locally.
- When files are copied with `COPY`, permissions are preserved.



Embedding unit tests in the build process

```
FROM <baseimage>
RUN <install dependencies>
COPY <code>
RUN <build code>
RUN <install test dependencies>
COPY <test data sets and fixtures>
RUN <unit tests>
FROM <baseimage>
RUN <install dependencies>
COPY <code>
RUN <build code>
CMD, EXPOSE ...
```

- The build fails as soon as an instruction fails
- If `RUN <unit tests>` fails, the build doesn't produce an image
- If it succeeds, it produces a clean image (without test libraries and data)



Dockerfile examples



There are a number of tips, tricks, and techniques that we can use in Dockerfiles.

But sometimes, we have to use different (and even opposed) practices depending on:

- the complexity of our project,
- the programming language or framework that we are using,
- the stage of our project (early MVP vs. super-stable production),
- whether we're building a final image or a base for further images,
- etc.

We are going to show a few examples using very different techniques.



When to optimize an image

When authoring official images, it is a good idea to reduce as much as possible:

- the number of layers,
- the size of the final image.

This is often done at the expense of build time and convenience for the image maintainer; but when an image is downloaded millions of times, saving even a few seconds of pull time can be worth it.

```
RUN apt-get update && apt-get install -y libpng12-dev libjpeg-dev && rm -rf \
/var/lib/apt/lists/* \
    && docker-php-ext-configure gd --with-png-dir=/usr --with-jpeg-dir=/usr \
    && docker-php-ext-install gd
...
RUN curl -o wordpress.tar.gz -SL https://wordpress.org/wordpress-
${WORDPRESS_UPSTREAM_VERSION}.tar.gz \
    && echo "$WORDPRESS_SHA1 *wordpress.tar.gz" | shasum -c - \
    && tar -xzf wordpress.tar.gz -C /usr/src/ \
    && rm wordpress.tar.gz \
    && chown -R www-data:www-data /usr/src/wordpress
```

(Source: [Wordpress official image](#))



When to *not* optimize an image

Sometimes, it is better to prioritize *maintainer convenience*.

In particular, if:

- the image changes a lot,
- the image has very few users (e.g. only 1, the maintainer!),
- the image is built and run on the same machine,
- the image is built and run on machines with a very fast link ...

In these cases, just keep things simple!

(Next slide: a Dockerfile that can be used to preview a Jekyll / github pages site.)



FROM debian:sid

RUN apt-get update -q

RUN apt-get install -yq build-essential make

RUN apt-get install -yq zlib1g-dev

RUN apt-get install -yq ruby ruby-dev

RUN apt-get install -yq python-pygments

RUN apt-get install -yq nodejs

RUN apt-get install -yq cmake

RUN gem install --no-rdoc --no-ri github-pages

COPY . /blog

WORKDIR /blog

VOLUME /blog/_site

EXPOSE 4000

CMD ["jekyll", "serve", "--host", "0.0.0.0", "--incremental"]



Multi-dimensional versioning systems

Images can have a tag, indicating the version of the image.

But sometimes, there are multiple important components, and we need to indicate the versions for all of them.

This can be done with environment variables:

```
ENV PIP=9.0.3 \
      ZC_BUILDOUT=2.11.2 \
      SETUPTOOLS=38.7.0 \
      PLONE_MAJOR=5.1 \
      PLONE_VERSION=5.1.0 \
      PLONE_MD5=76dc6cfc1c749d763c32fff3a9870d8d
```

(Source: [Plone official image](#))

It is very common to define a custom entrypoint.

That entrypoint will generally be a script, performing any combination of:

- pre-flights checks (if a required dependency is not available, display a nice error message early instead of an obscure one in a deep log file),
- generation or validation of configuration files,
- dropping privileges (with e.g. `su` or `gosu`, sometimes combined with `chown`),
- and more.

A typical entrypoint script

```
#!/bin/sh
set -e

# first arg is '-f' or '--some-option'
# or first arg is 'something.conf'
if [ "${1#-}" != "$1" ] || [ "${1%.conf}" != "$1" ]; then
    set -- redis-server "$@"
fi

# allow the container to be started with '--user'
if [ "$1" = 'redis-server' -a "$(id -u)" = '0' ]; then
    chown -R redis .
    exec su-exec redis "$0" "$@"
fi

exec "$@"
```

(Source: [Redis official image](#))



Factoring information

To facilitate maintenance (and avoid human errors), avoid to repeat information like:

- version numbers,
- remote asset URLs (e.g. source tarballs) ...

Instead, use environment variables.

```
ENV NODE_VERSION 10.2.1
...
RUN ...
    && curl -fsSLO --compressed "https://nodejs.org/dist/v$NODE_VERSION/node-
v$NODE_VERSION.tar.xz" \
    && curl -fsSLO --compressed
"https://nodejs.org/dist/v$NODE_VERSION/SHASUMS256.txt.asc" \
    && gpg --batch --decrypt --output SHASUMS256.txt SHASUMS256.txt.asc \
    && grep " node-v$NODE_VERSION.tar.xz\$" SHASUMS256.txt | sha256sum -c - \
    && tar -xf "node-v$NODE_VERSION.tar.xz" \
    && cd "node-v$NODE_VERSION" \
...
```

(Source: [Nodejs official image](#))



In theory, development and production images should be the same.

In practice, we often need to enable specific behaviors in development (e.g. debug statements).

One way to reconcile both needs is to use Compose to enable these behaviors.

Let's look at the [trainingwheels](#) demo app for an example.

This Dockerfile builds an image leveraging gunicorn:

```
FROM python
RUN pip install flask
RUN pip install gunicorn
RUN pip install redis
COPY . /src
WORKDIR /src
CMD gunicorn --bind 0.0.0.0:5000 --workers 10
    counter:app
EXPOSE 5000
```

(Source: [trainingwheels Dockerfile](#))

Development Compose file

This Compose file uses the same image, but with a few overrides for development:

- the Flask development server is used (overriding `CMD`),
- the `DEBUG` environment variable is set,
- a volume is used to provide a faster local development workflow.

```
services:
  www:
    build: www
    ports:
      - 8000:5000
    user: nobody
    environment:
      DEBUG: 1
    command: python counter.py
    volumes:
      - ./www:/src
```

(Source: [trainingwheels Compose file](#))



How to know which best practices are better?

- The main goal of containers is to make our lives easier.
- In this chapter, we showed many ways to write Dockerfiles.
- These Dockerfiles use sometimes diametrically opposed techniques.
- Yet, they were the “right” ones *for a specific situation*.
- It’s OK (and even encouraged) to start simple and evolve as needed.
- Feel free to review this chapter later (after writing a few Dockerfiles) for inspiration!

Let's update our Dockerfiles to leverage multi-stage builds!

The code is at: <https://github.com/jpetazzo/wordsmith>

Use a different tag for these images, so that we can compare their sizes.

What's the size difference between single-stage and multi-stage builds?

